INTERPROCESS COMMUNICATION AND SYNCHRONIZATION

COM3306

H.M.C.J.DE SILVA

DEPARTMENT OF COMPUTING

ILO'S AT THE END OF THE LESSON

- ILO 1: Explain the purpose of Interprocess communication (IPC) in operating systems.
- ILO 2: Describe various IPC mechanisms, including pipes, message queues, shared memory, and sockets.
- ILO 3: Define synchronization and explain why it is necessary in concurrent systems.
- ILO 4: Analyze synchronization problems like race conditions, critical sections, and deadlocks.
- ILO 5: Differentiate between various IPC mechanisms in terms of use cases, performance, and complexity.
- ILO 6: Apply synchronization primitives (e.g., semaphores, mutexes, monitors) to solve concurrency issues in programs.

INTRODUCTION TO PROCESSES AND THREADS

- **Process** : Instance of a program in execution, independent execution units.
- **Thread** : Lightweight processes within a process, sharing memory but can run concurrently.
- **Importance of Process Communication and Synchronization**: Ensures smooth and efficient operation of multi-process/multi-thread systems.

INTERPROCESS COMMUNICATION(IPC)

- Interprocess communication (IPC) involves sending information from one process to another.
- The processes can be running on the same computer or on different computers connected through a network.
- IPC enables one application to control another application, and for several applications to share the same data without interfering with one another.
- IPC is required in all multiprogramming systems, but it is not generally supported by single process operating systems such as DOS.

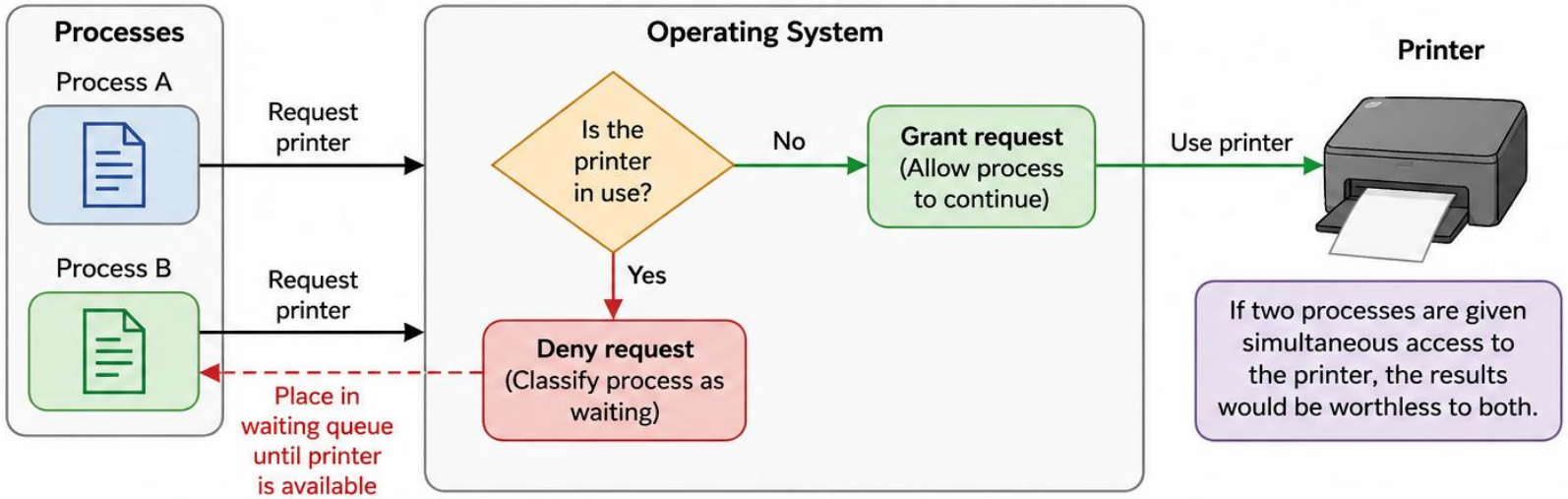
INTERPROCESS COMMUNICATION(IPC)

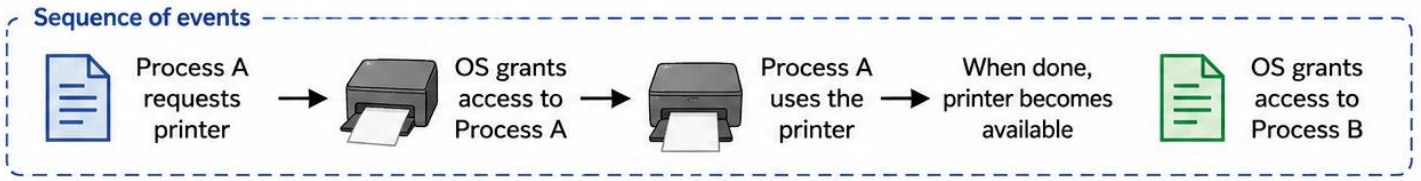
- Example

Consider a machine with a single printer running a time-sharing operation system. If a process needs to print its results, it must request that the operating system to give it access to the printer's device driver. At this point, the operating system must decide whether to grant this request, depending upon whether the printer is already being used by another process. If it is not, the operating system should grant the request and allow the process to continue; otherwise, the operating system should deny the request and perhaps classify the process as a waiting process until the printer becomes available. Indeed, if two processes were given simultaneous access to the machine's printer, the results would be worthless to both.

- **Example**

Consider a machine with a single printer running a time-sharing operation system. If a process needs to print its results, it must request that the operating system to give it access to the printer's device driver. At this point, the operating system must decide whether to **grant this request**, depending upon whether the printer is already being used by another process. If it is not, the operating system should grant the request and **allow the process to continue**; otherwise, the operating system should **deny the request** and perhaps classify the process as a **waiting process** until the printer becomes available. Indeed, if two processes were given simultaneous access to the machine's printer, the results would be worthless to both.





INTERPROCESS COMMUNICATION(IPC)

- There exist two complementary inter-process communication types:

  a) shared-memory system

  b) message-passing system

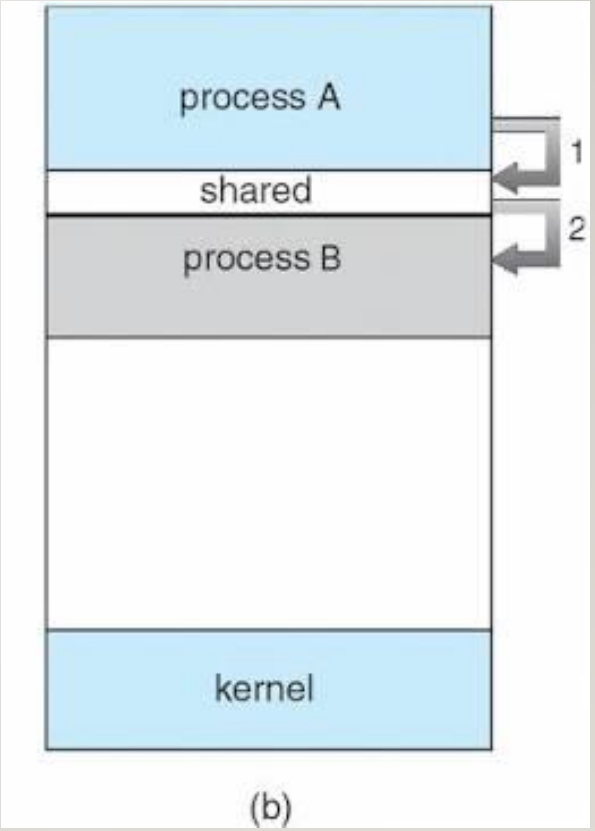
  c)Pipes

  4)Sockets

  5)Remote Procedure Calls(RPC)

SHARED MEMORY SYSTEM

- Require communication processes to share some variables.
- Processes exchange information through the use of shared variables.
- In a shared-memory scheme, the responsibility for providing communication rests with the application programmers. The operating system only needs to provide shared memory.



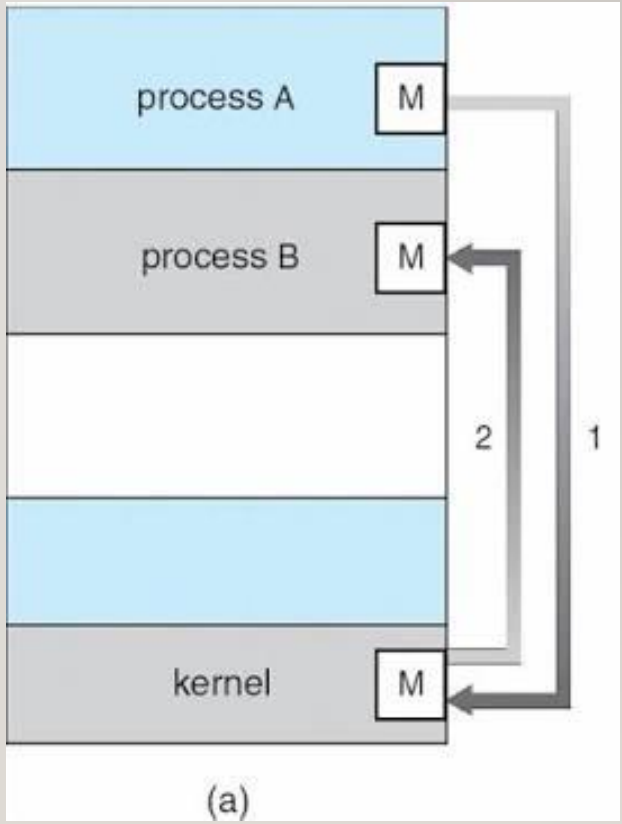
MESSAGE PASSING SYSTEM

- Allow communication processes to exchange messages. In this scheme responsibility rests with OS itself.
- An Interprocess communication facility basically provides two operations: send (message) and receive (message). In order to send and to receive messages, a communication link must exist between two involved processes.
- Message passing can happen in two ways:
  - Direct communication

  In direct communication, each process that wants to send or receive a message must explicitly name the recipient or sender of the communication. In this case, the send and receive primitives are defined as follows:

  send (P, message) ->To send a message to process P

  receive (Q, message) -> To receive a message from process Q



MESSAGE PASSING SYSTEM

- Indirect communication

  Messages are sent to and received from a mailbox. In order to distinguish one from the other, each mailbox owns a unique identification. A process may communicate with some other process by a number of different mailboxes that owned by either by a process or by a system. The send and receive primitives are defined as follows:

  send (A, message) -> To send a message to the mailbox A

  receive (A, message) -> To receive a message from the mailbox A

- There are three capacities, of links that determines number of messages attached to the link.

MESSAGE PASSING SYSTEM

1. Zero capacity-Link has a message queue length of zero, i.e., no message can wait in it. The sender must wait until the recipient receives the message.
2. Bounded capacity-This link has a limited message queue length of n, i.e., at most n messages can reside in it. If a new message is sent, and the queue is not full, it is placed in the queue either by copying the message or by keeping a pointer to message and the sender should continue execution without waiting. Otherwise, the sender must be delayed until space is available in the queue
3. Unbounded capacity-This queue has potentially infinite length, i.e., any number of messages can wait in it. That is why the sender is never delayed.

PROCESS SYNCHRONIZATION IN OS

"Preserving the appropriate order of execution of cooperative processes or threads so they do not interfere with each other while accessing shared resources is known as Process Synchronization."

- Cooperative processes- Process whose execution can affect or be affected by execution of another process

PROCESS SYNCHRONIZATION IN OS

- Why is it necessary?
  - Many processes or threads run at the same time.
  - They share data or resources like files, variables in memory, printers and etc.
  - If multiple processes modify shared data at the same time, errors can happen as:
    - They overwrite each other's changes
    - The result becomes unpredictable

    ex-Bank account example:

PROCESS SYNCHRONIZATION IN OS

- Goals of synchronization
  - Ensure data consistency.
  - Prevent race conditions.
  - Control the order of execution when needed.

PROBLEMS ARISE WITHOUT PROPER SYNCHRONIZATION

- In a multi-tasking system, processes or threads often share resources like memory, files, and variables. Without synchronization, the following issues can arise:

1. Race Conditions

- Occurs when two or more processes access shared data simultaneously and try to change it.
- The final outcome depends on who finishes first.
- Unpredictable results.
- Ex-Two threads updating the same variable-Result depends on timing

2. Data Inconsistency:

- Shared resources can become inconsistent if multiple processes modify them at the same time.

PROBLEMS ARISE WITHOUT PROPER SYNCHRONIZATION

3.Deadlocks

- Happens when two or more processes wait for resources held by each other
- None of them can proceed
- System becomes stuck forever.

4.Starvation

- A process never gets the necessary resources to proceed due to others taking priority.

RACE CONDITION AND CRITICAL SECTION

- A **Race Condition** typically occurs when two or more threads try to read, write, and possibly make the decisions based on the memory they are accessing concurrently.(occur when two computer program processes, or threads, attempt to access the same resource at the same time and cause problems in the system)
- Example: Shared printer

RACE CONDITION AND CRITICAL SECTION

- The region of a program that tries to access shared resources and may cause race condition is called a **critical section**.
- To avoid race conditions among the processes, we must ensure that only one process can be executed within the critical section at a time.
- Shared resources can be:
  - Memory locations – Shared variable count
  - Files and file systems – Two programs writing to same log file
  - CPU- Two threads competing for CPU to run their tasks
  - I/O devices- Two processes trying to print at same time
  - Databases- Two transactions updating the same account balance.

RACE CONDITION AND CRITICAL SECTION

- The critical section problem is used to design a set of protocols which can ensure that the Race condition among the processes will never arise.

REQUIREMENTS OF SYNCHRONIZATION MECHANISMS

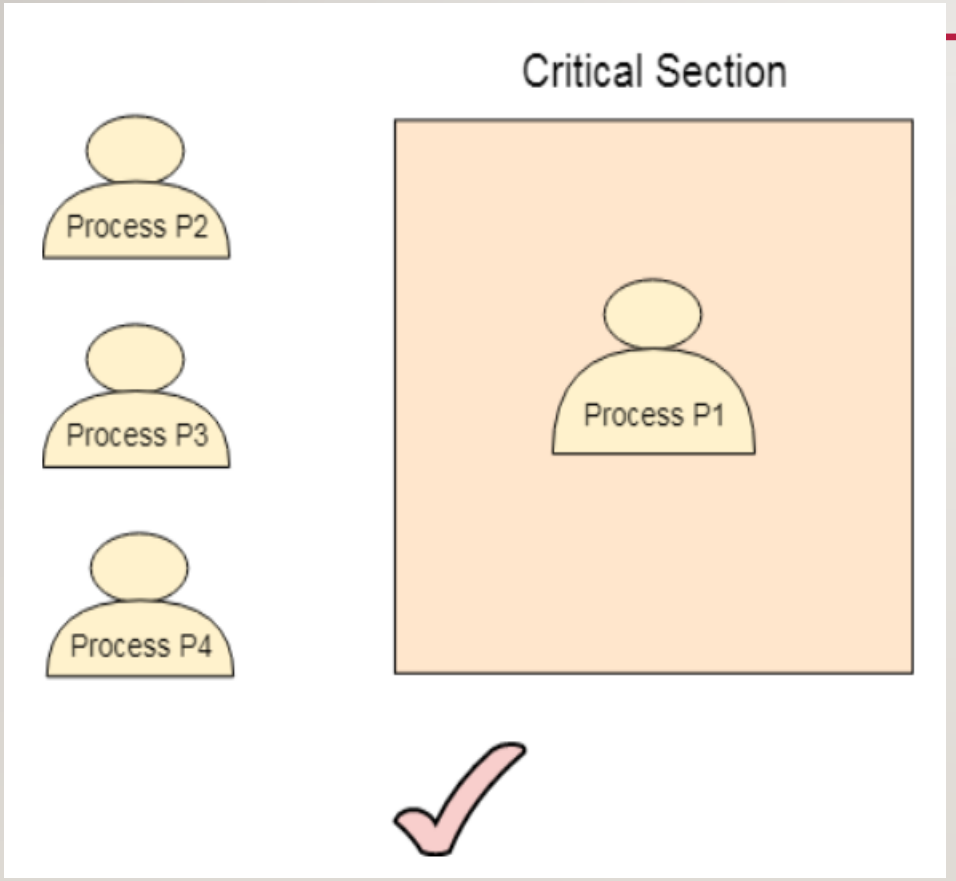
1. Mutual Exclusion

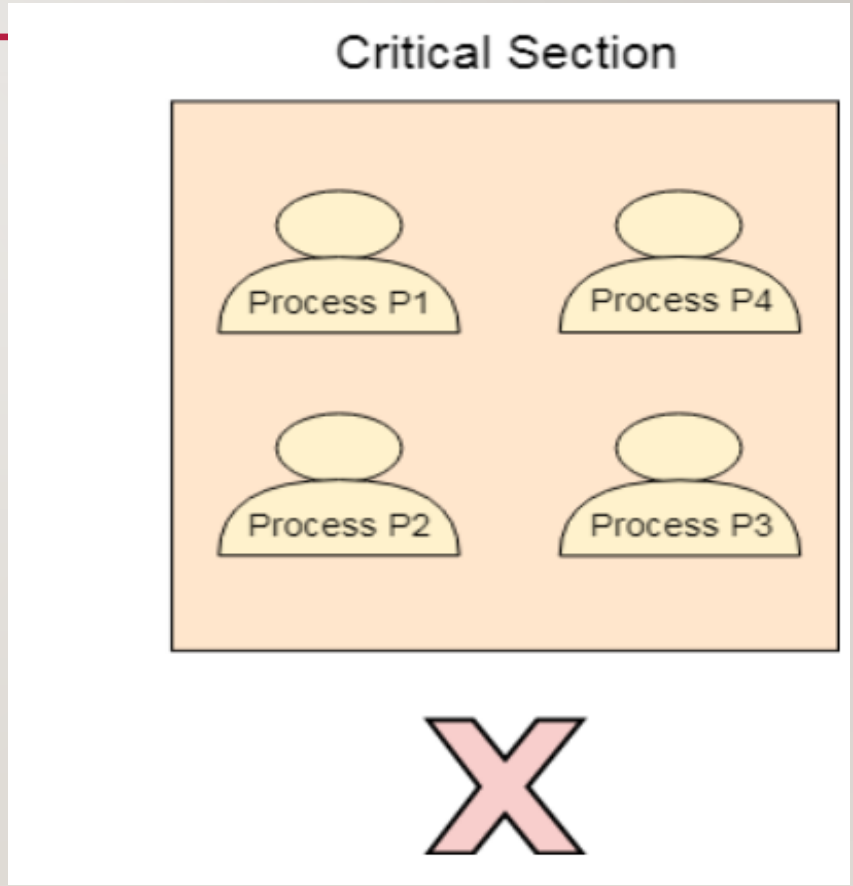
If one process is executing inside a critical section then the other process must not enter the critical section. This is called Mutual Exclusion.

Why?

To avoid conflicts when accessing shared resources.

Example: Shared printer- If two processes send print jobs at the same time pages mix up.


Critical Section
Process P2
Process P3
Process P4
Process P1


Critical Section
Process P1
Process P4
Process P2
Process P3

REQUIREMENTS OF SYNCHRONIZATION MECHANISMS

2. Progress

If one process doesn't need to execute into a critical section, then it should not stop other processes from getting into the critical section. OS should not make processes wait forever unnecessarily.

Example-

Two processes want to write to a shared file. No one is inside the critical section. Progress says let one go in.

REQUIREMENTS OF SYNCHRONIZATION MECHANISMS

3. Bounded waiting

We should be able to predict the waiting time for every process to get into the critical section. The process must not be endlessly waiting for getting into the critical section.

Example-

Process A wants to enter the critical section. Bounded waiting ensures:

Process A will eventually get its turn.

Other processes cannot “jump the queue” endlessly.

LOCK VARIABLE MECHANISM

- The **lock variable mechanism** is one of the earliest, simplest ways to **prevent race conditions** in operating systems.
- It tries to ensure **mutual exclusion** — i.e. only one process or thread is allowed in the critical section at a time.
- This is a busy waiting solution that can be used for more than two processes.
- It uses a shared variable often called **Lock**
- Typical lock variable values are:
  - **0 -> Critical section is free**
  - **1-> Critical section is occupied**

LOCK VARIABLE MECHANISM

- A process that wants to enter the critical section:
  1. **Checks the lock variable**
     1. **If lock==0, the critical section is free**
  2. **Sets the lock to 1 -> Reserve the critical section**
  3. **Execute the critical section**
  4. **Sets the lock back to 0-> Release the critical section**

LOCK VARIABLE

Entry Section →

```
While (lock! = 0) ;
Lock = 1;
//Critical Section
```

Exit Section →

```
Lock =0;
```

- Pseudocode consists of 3 sections as entry section, the critical section and the exit section.

LOCK VARIABLE

- The problem with the lock variable mechanism is that, at the same time, more than one process can see the vacant tag and more than one process can enter in the critical section.
- Hence, the lock variable doesn't provide mutual exclusion that's why it cannot be used in general.
- Since the lock variable method failed at the basic step; hence, there is no need to talk about the other conditions to be fulfilled.
    1) Load Lock, R0
    2) CMP R0,#0
    3) JNZ step 1
    4) Store #1,lock

TEST SET LOCK MECHANISMS

- A hardware solution to the synchronization problem.
- There is a **shared lock variable** which can take either of two values **0 (unlock) or 1 (locked)**.
- Before entering to critical section, process inquires about the lock whether it Is 0 or 1.This is called as ‘Testing part’.
- If it is locked, it keeps on waiting till it became free.
- If it is not locked, it takes the lock and execute the critical section.

TEST SET LOCK MECHANISMS

Example with Two processes (P1 and P2)

- Initially, lock = FALSE
- P1 executes Test-and-Set → gets FALSE → enters critical section
- P2 executes Test-and-Set → gets TRUE → waits
- P1 finishes and sets lock = FALSE
- P2 now gets the lock and enters

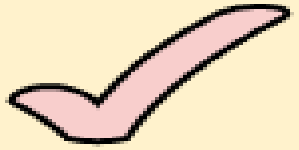
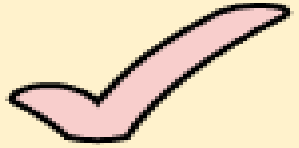
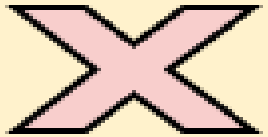
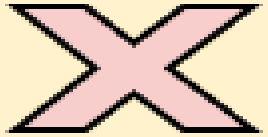
TEST SET LOCK MECHANISMS

- The Test-and-Set lock mechanism uses a special atomic hardware instruction. The instruction checks and sets the lock in one indivisible step.
- Test-and-Set guarantees mutual exclusion.

Problems with Test set Lock:

- Busy Waiting-Processes continuously check the lock in a loop. This wastes CPU time.
- Starvation-A process may wait indefinitely if others repeatedly acquire the lock.
- Priority Inversion Low-priority process may hold the lock while high-priority process waits.

TEST SET LOCK MECHANISM

Mutual Exclusion	
Progress	
Bounded Waiting	
Portability	

TURN VARIABLE OR STRICT ALTERATION APPROACH

- Turn Variable or Strict Alternation Approach is the software mechanism implemented at user mode.
- It is a busy waiting solution which can be implemented only for two processes.
- They share a variable called turn variable.
- It allows two processes to access a critical section alternately. Only one process whose turn it is can enter the critical section.
- If two processes are Pi and Pj
  - If turn=i, process Pi can enter the critical section
  - If turn =j ,process Pj can enter the critical section

TURN VARIABLE OR STRICT ALTERATION APPROACH

For process Pi

```
while (turn != i);   // wait
/* critical section */
turn = j;
/* remainder section */
```

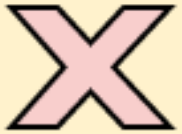
For process Pj

```
while (turn != j);   // wait
/* critical section */
turn = i;
/* remainder section */
```

TURN VARIABLE OR STRICT ALTERATION APPROACH

- Problems with this approach are:
  - Violation of progress condition - A process must wait even if the other process does not want to enter the critical section. This causes unnecessary waiting.
  - Busy waiting - Processes repeatedly check the turn variable in a loop. This wastes CPU time.
  - Limited to two processes

TURN VARIABLE OR STRICT ALTERATION APPROACH

Mutual Exclusion	
Progress	
Bounded Waiting	
Portability	

PETERSON'S SOLUTION

- This is a software mechanism implemented at user mode. It is a busy waiting solution can be implemented for only two processes.
- It uses two shared variables that are **flag** variable and **turn** variable.
  - Flag array: Indicates whether a process wants to enter the critical section.

    Boolean array of size 2, where flag[i] = true indicates process Pi wants to enter to critical section
  - Turn variable: Integer variable to determine which process has "turn" to enter critical section.
- Peterson solution provides you all the necessary requirements such as Mutual Exclusion, Progress, Bounded Waiting.

PETERSON'S SOLUTION- FOR PROCESS PI(I=0 OR 1)

```
flag[i] = true;        // Pi wants to enter
turn = j;              // give turn to other process

while (flag[j] && turn == j);
/* critical section */

flag[i] = false;       // Pi leaves critical section
/* remainder section */
```

PETERSON'S SOLUTION

1. Entering the Critical Section:

- A process wishing to enter the critical section sets its corresponding flag to true.
- It then sets the turn variable to the other process's ID, indicating its willingness to yield the turn.
- It enters a while loop that continues as long as the other process's flag is true and it's the other process's turn. This loop effectively waits until the other process is done or yields the turn.

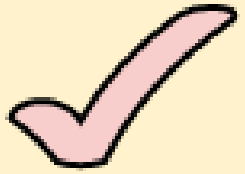
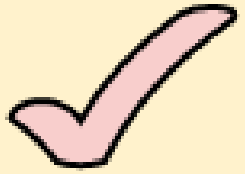
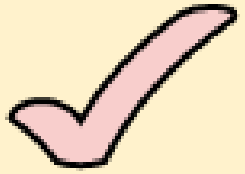
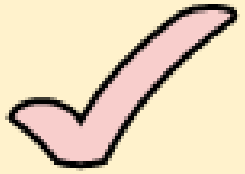
PETERSON'S SOLUTION

2.Inside the Critical Section: The process can now safely access the shared resource.

3.Exiting the Critical Section:

- The process resets its flag to false, indicating it no longer needs the critical section.
- This allows the other process (if waiting) to potentially enter the critical section.

PETERSON'S SOLUTION

Mutual Exclusion	
Progress	
Bounded Waiting	
Portability	

SLEEP AND WAKE APPROACH TO SOLVE-PRODUCER CONSUMER PROBLEM

- The concept of sleep and wake is if the critical section is not empty then the process will go and sleep. It will be waked up by the other process which is currently executing inside the critical section so that the process can get inside the critical section.
- The goal is to make sure that both processes don't interfere with each other while accessing a shared resource, which is a buffer in this case. The producer creates items, and the consumer takes them..
- In order to read and write, both of them are using a buffer.

SLEEP AND WAKE-PRODUCER CONSUMER PROBLEM

- Key elements of the problem:
- **Producer:** A process that generates data and places it into the shared buffer.
- **Consumer:** A process that retrieves data from the buffer and uses it.
- **Buffer:** A shared data structure (often a queue or array) with a fixed size, used to store items produced by the producer before being consumed.
- **Synchronization:** Mechanisms (like semaphores) are needed to ensure that the producer doesn't try to write to a full buffer, and the consumer doesn't try to read from an empty buffer.

SLEEP AND WAKE-PRODUCER CONSUMER PROBLEM

- The producer produces the item and inserts it into the buffer. The value of the global variable count got increased at each insertion.
- If the buffer is filled completely and no slot is available then the producer will sleep, otherwise it keep inserting.
- On the consumer's end, the value of count got decreased by 1 at each consumption.
- If the buffer is empty at any point of time then the consumer will sleep otherwise, it keeps consuming the items and decreasing the value of count by 1.
- The consumer will be waked up by the producer if there is at least 1 item available in the buffer which is to be consumed. The producer will be waked up by the consumer if there is at least one slot available in the buffer so that the producer can write that.

SLEEP AND WAKE-PRODUCER CONSUMER PROBLEM

- **Producer Process**
- Checks if the buffer is full.
- If the buffer is full, the producer calls sleep() and waits.
- If space is available, the producer adds an item to the buffer.
- After producing, it calls wakeup(consumer).
- **Consumer Process**
- Checks if the buffer is empty.
- If the buffer is empty, the consumer calls sleep() and waits.
- If items are available, the consumer removes an item.
- After consuming, it calls wakeup(producer).

SLEEP AND WAKE-PRODUCER CONSUMER PROBLEM

- Lost Wakeup Problem
  - The producer may call wakeup(consumer) before the consumer actually goes to sleep
  - The wakeup signal is lost
  - Consumer sleeps forever → deadlock

SLEEP AND WAKE-PRODUCER CONSUMER PROBLEM

Using a flag bit to get rid of this problem

- A flag bit can be used in order to get rid of this problem.
- The producer can set the bit when it calls wake-up on the first time. When the consumer got scheduled, it checks the bit.
- The consumer will now get to know that the producer tried to wake him and therefore it will not sleep and get into the ready state to consume whatever produced by the producer.
- This solution works for only one pair of producer and consumer.

Thank You!